```
==================== Program Code ======================
import numpy as np
from scipy.linalg import lu
from sympy import Matrix
import time

# Task 1: Generate matrix A and vector b
print("Step 1:")
A = np.array([
    [17, 24,  1,  8, 15],
    [23,  5,  7, 14, 16],
    [ 4,  6, 13, 20, 22],
    [10, 12, 19, 21,  3],
    [11, 18, 25,  2,  9]
], dtype=float)

b = np.array([10, 26, 42, 59, 38], dtype=float)

print("A =")
print(A)
print("\nb =", b)

# Task 2: Solve using numpy's built-in solver
print("\nStep 2:")
x = np.linalg.solve(A, b)
print("x =", x)

# Task 3: Compute the residual
print("\nStep 3:")
r = A @ x - b
print("r =", r)

# Task 4: LU Decomposition
print("\nStep 4:")
P, L, U = lu(A)
x1 = np.linalg.solve(U, np.linalg.solve(L, P.T @ b))
r1 = A @ x1 - b
err1 = x - x1
print("x1 =", x1)
print("r1 =", r1)
print("err1 =", err1)

# Task 5: Solving yA = b (equivalent to MATLAB's b'/A)
print("\nStep 5:")
y = np.linalg.solve(A.T, b)
print("y =", y)

# Task 6: Solving using explicit inverse matrix
print("\nStep 6:")
x2 = np.linalg.inv(A) @ b
r2 = A @ x2 - b
err2 = x - x2
print("x2 =", x2)
print("r2 =", r2)
print("err2 =", err2)

# Task 7: Reduced row echelon form
print("\nStep 7:")
augmented = Matrix(np.hstack([A, b.reshape(-1, 1)]))
C, pivot_cols = augmented.rref()
C = np.array(C.tolist(), dtype=float)
x3 = C[:, 5]
err3 = x - x3
r3 = A @ x3 - b
print("x3 =", x3)
print("err3 =", err3)
print("r3 =", r3)
```

```
# Task 8: Computational time comparison
print("\nStep 8:")
Num = 100
A_big = np.random.rand(Num, Num) + Num * np.eye(Num)
b_big = np.random.rand(Num)

def numerical_rref(M):
    M = M.copy()
    rows, cols = M.shape
    pivot_row = 0
    for col in range(cols):
        if pivot_row >= rows:  # <-- add this check
            break
        pivot = np.argmax(np.abs(M[pivot_row:, col])) + pivot_row
        if abs(M[pivot, col]) < 1e-10:
            continue
        M[[pivot_row, pivot]] = M[[pivot, pivot_row]]
        M[pivot_row] /= M[pivot_row, col]
        for r in range(rows):
            if r != pivot_row:
                M[r] -= M[r, col] * M[pivot_row]
        pivot_row += 1
    return M

print("Matrix size %d x %d" % (Num, Num))

t0 = time.time()
x1_big = np.linalg.solve(A_big, b_big)
print(f"np.linalg.solve: {time.time() - t0:.4f} seconds")

t0 = time.time()
x2_big = np.linalg.inv(A_big) @ b_big
print(f"np.linalg.inv:   {time.time() - t0:.4f} seconds")

t0 = time.time()
aug = np.hstack([A_big, b_big.reshape(-1,1)])
C_big = numerical_rref(aug)
print(f"rref:            {time.time() - t0:.4f} seconds")

# Task 9: Overdetermined system
print("\nStep 9:")
clear = lambda: None  # no need to clear in Python
A9 = np.array([[1,2,3],[4,5,6],[7,8,10],[9,11,12]], dtype=float)
b9 = np.array([1,2,3,4], dtype=float)

x, residuals, rank, sv = np.linalg.lstsq(A9, b9, rcond=None)
r = A9 @ x - b9
print("x =", x)
print("r =", r)

# Verify via normal equations
y = np.linalg.solve(A9.T @ A9, A9.T @ b9)
err = x - y
print("y =", y)
print("err =", err)

# Task 10: Underdetermined system
print("\nStep 10:")
A10 = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12]], dtype=float).T
b10 = np.array([1,3,5], dtype=float)

# Particular solution from lstsq
x, _, _, _ = np.linalg.lstsq(A10, b10, rcond=None)
r1 = A10 @ x - b10
print("x =", x)
print("r1 =", r1)

# Minimum-norm solution via pseudoinverse
```

```
y = np.linalg.pinv(A10) @ b10
r2 = A10 @ y - b10
print("y =", y)
print("r2 =", r2)

# Null space (homogeneous solutions)
from scipy.linalg import null_space
N = null_space(A10)
print("Null space basis:\n", N)

# General solution: pick any constants C1, C2
C = np.array([1, 2])
z = N @ C + x
print("z =", z)
print("Verify Az=b:", np.allclose(A10 @ z, b10))

==================== Program Output ====================
Step 1:
A =
[[17. 24.  1.  8. 15.]
 [23.  5.  7. 14. 16.]
 [ 4.  6. 13. 20. 22.]
 [10. 12. 19. 21.  3.]
 [11. 18. 25.  2.  9.]]

b = [10. 26. 42. 59. 38.]

Step 2:
x = [-0.00833333  0.0724359   1.47179487  1.48782051 -0.33141026]

Step 3:
r = [3.55271368e-15 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00]

Step 4:
x1 = [-0.00833333  0.0724359   1.47179487  1.48782051 -0.33141026]
r1 = [3.55271368e-15 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00]
err1 = [0. 0. 0. 0. 0.]

Step 5:
y = [ 0.18237179 -0.525       1.86923077  1.325      -0.15929487]

Step 6:
x2 = [-0.00833333  0.0724359   1.47179487  1.48782051 -0.33141026]
r2 = [ 3.55271368e-15 -3.55271368e-15  0.00000000e+00 -2.13162821e-14
  7.10542736e-15]
err2 = [ 2.93168267e-16 -1.38777878e-17  0.00000000e+00  6.66133815e-16
 -8.32667268e-16]

Step 7:
x3 = [-0.00833333  0.0724359   1.47179487  1.48782051 -0.33141026]
err3 = [ 2.25514052e-17 -4.16333634e-17  0.00000000e+00 -2.22044605e-16
  3.33066907e-16]
r3 = [0. 0. 0. 0. 0.]

Step 8:
Matrix size 100 x 100
np.linalg.solve: 0.0003 seconds
np.linalg.inv:   0.0010 seconds
rref:            0.0276 seconds

Step 9:
x = [-0.24630542  0.38916256  0.16256158]
r = [ 0.01970443 -0.06403941  0.01477833  0.01477833]
y = [-0.24630542  0.38916256  0.16256158]
err = [ 1.92901251e-14 -1.76525461e-14  1.47104551e-15]
```

```
Step 10:
x = [ 1.5         0.83333333  0.16666667 -0.5       ]
r1 = [5.32907052e-15 4.44089210e-15 3.55271368e-15]
y = [ 1.5         0.83333333  0.16666667 -0.5       ]
r2 = [4.44089210e-15 3.55271368e-15 8.88178420e-16]
Null space basis:
 [[-0.47518507 -0.27239522]
 [ 0.43054868  0.71737566]
 [ 0.56445783 -0.61756567]
 [-0.51982145  0.17258523]]
z = [ 0.4800245   2.69863333 -0.50400683 -0.674651  ]
Verify Az=b: True
```